Optimización en tiempo real de un pipeline de detección YOLO: medición antes y después

Actividad 4, Unidad 3 · Técnicas Avanzadas de IA · Grupo 9
Alejandro Moncada · Edgar Julián Méndez Ortegón · Noel Eduardo Pérez Barrios · 12 de octubre de 2026
Código: github.com/AlejoMoncada/yolo-realtime-pipeline

1. Objetivo y resultado

Se midió el pipeline de detección sin optimizar y después optimizado, con el mismo video, el mismo equipo y los mismos parámetros, separando captura, preprocesado, inferencia, cola, anotación y visualización. **Resultado:** el rendimiento anotado pasa de **49,1 FPS** (síncrono, PyTorch, 640 px) a **138,3 FPS** (hilos + colas, ONNX Runtime, 320 px), estable entre 104 y 147 FPS por ventana de un segundo; en pantalla real el render pasa de **26,9** a **73,8 FPS**. Al ritmo de una cámara (12,5 FPS), la latencia de extremo a extremo p95 baja de **34,4 ms** a **13,6 ms**. El salto proviene de reducir el costo de inferencia (menor resolución y formato ONNX); la arquitectura concurrente desacopla captura, inferencia y visualización sin condiciones de carrera, pero a 640 px no aumenta el rendimiento por sí sola.

2. Arquitectura concurrente

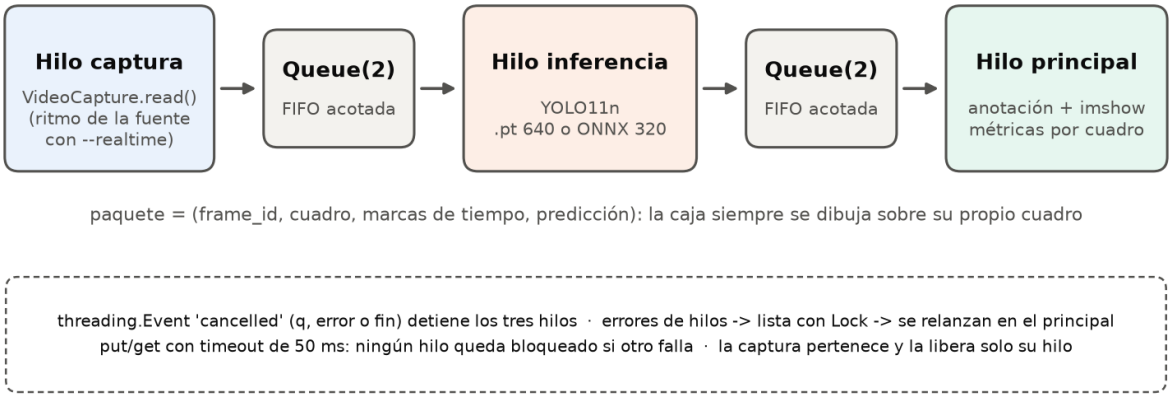


Figura 1. Flujo de hilos del modo optimizado (vision_pipeline.run_threaded).

Tres hilos se comunican solo mediante dos `queue.Queue(maxsize=2)`, que son seguras entre hilos; no hay estado mutable compartido fuera de ellas, por lo que no existen condiciones de carrera sobre cuadros o cajas. Cada paquete viaja con su `frame_id`: la anotación se dibuja siempre sobre el cuadro que la produjo, así que las cajas nunca quedan desfasadas respecto de la imagen. La cola acotada aplica contrapresión: si la inferencia es más lenta, la captura espera en lugar de acumular memoria. Un `threading.Event` cancela todos los hilos ante la tecla q, un error o el fin del video, y los errores de cualquier hilo se propagan al hilo principal. Este comportamiento está cubierto por 47 pruebas automáticas (orden, emparejamiento, cancelación, errores y liberación de la captura).

3. Protocolo de medición

- **Entrada:** car-detection.mp4, 768×432, 12,5 FPS, 377 cuadros; SHA-256 d31e0ebf194cc16e… registrado en cada comparison.json. Todos los casos procesan exactamente los mismos frame_id.
- **Equipo:** Apple M4 (10 núcleos, 16 GB), macOS, solo CPU (device=cpu). Python 3.12.11, Ultralytics 8.4.176, PyTorch 2.14.1, OpenCV 4.14.0, ONNX Runtime 1.31.0, modelo yolo11n, conf=0,25, batch=1.
- **Casos:** antes = síncrono, .pt, 640 px; intermedio = hilos, .pt, 640 px (aísla el efecto de la concurrencia); después = hilos, ONNX exportado con tamaño fijo, 320 px.
- **Repetición:** 3 corridas por caso, orden rotado entre repeticiones; 2 inferencias de calentamiento fuera de la medición; se reporta la mediana entre corridas.
- **Tres escenarios:** (a) offline: el archivo se lee lo más rápido posible y mide rendimiento máximo; (b) --realtime: cada cuadro se libera en origen + n/12,5 s, como una cámara, y la latencia se mide desde esa llegada (incluye atrasos); (c) --show: visualización real con cv2.imshow, una corrida por caso, para medir FPS de render.

- **Métricas por cuadro:** captura, preproceso/inferencia/postproceso (Results.speed de Ultralytics), espera en colas, anotación, visualización y E2E (media, p50, p95, p99); FPS por ventana de 1 s para estabilidad; CPU del proceso con psutil (100 % = un núcleo, por eso supera 100 %).

4. Resultados antes y después

Caso (offline)	FPS (med.)	Cap-tura	Pre-proc.	Infe-rencia	Post-proc.	Cola	Anota-ción	E2E p50 / p95 / p99	CPU	FPS por seg. (mín–máx)
Antes: síncrono, .pt, 640	49,1	0,18	0,51	18,00	0,19	0,00	0,09	19,2 / 27,3 / 33,3	233 %	36–51
Hilos, .pt, 640	42,4	0,28	0,72	20,42	0,21	65,67	0,16	88,3 / 126,9 / 204,7	231 %	30–54
Después: hilos, ONNX, 320	138,3	0,43	0,83	5,52	0,28	20,18	0,22	27,7 / 37,2 / 40,5	491 %	104–147

Tabla 1. Escenario offline (mediana de p50 por etapa en ms; E2E en ms). Corrida benchmark-20261012-171741-64736e26.

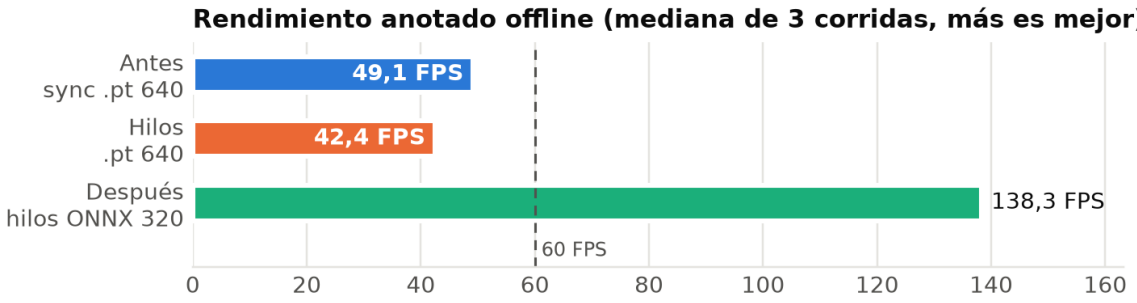


Figura 2. Solo el caso optimizado supera 60 FPS; los hilos por sí solos a 640 px no aumentan el rendimiento.

Caso (--realtime, 12,5 FPS)	Inferencia p50	Cola p50	Retraso de captura p95	E2E p50	E2E p95	E2E p99	CPU
Antes: síncrono, .pt, 640	20,34	0,00	0,00	24,9	34,4	61,6	69 %
Hilos, .pt, 640	21,30	3,61	0,00	26,0	45,3	112,7	72 %
Después: hilos, ONNX, 320	5,62	3,92	0,00	10,8	13,6	19,0	85 %

Tabla 2. Escenario al ritmo de cámara (ms). Los tres casos procesan los 12,5 FPS sin atraso acumulado. Corrida benchmark-20261012-172526-e682279c.

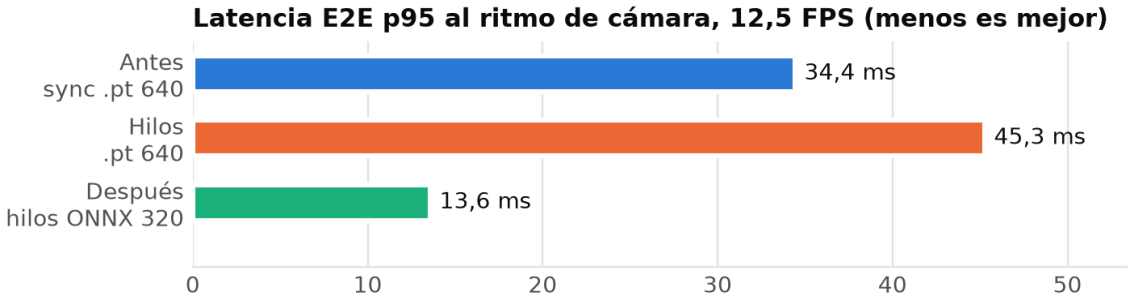


Figura 3. Con la fuente a ritmo real, el caso optimizado reduce la latencia p95 a menos de la mitad.

Caso (--show, pantalla real)	FPS de render	Visualización p50	E2E p95	FPS por seg. (mín–máx, desv.)
Antes: síncrono, .pt, 640	26,9	12,2 ms	45,5 ms	24–31 (2,2)
Después: hilos, ONNX, 320	73,8	12,2 ms	91,8 ms	52–80 (10,8)

Tabla 3. Render en pantalla (una corrida por caso, video leído sin límite de ritmo).

La E2E p95 del caso optimizado en esta tabla (91,8 ms) no indica que la visualización empeore la latencia: como el video se lee sin límite de ritmo, la cola acotada permanece llena y cada cuadro espera su turno (espera en cola p50 de unos 66 ms). Es contrapresión de la prueba de rendimiento máximo; la latencia que percibiría un usuario con una cámara es la de la Tabla 2 (13,6 ms).

5. Justificación de las decisiones por comportamiento de CPU y GPU

- **El cuello de botella es la inferencia.** En el caso síncrono la inferencia ocupa 94 % del tiempo por cuadro; la captura (0,18 ms) y la anotación (0,09 ms) son despreciables. Por eso los hilos a 640 px no aumentan el rendimiento (42,4 frente a

49,1 FPS): no hay trabajo comparable que solapar.

- **Los hilos sí aportan desacople.** PyTorch y ONNX Runtime liberan el GIL durante el cálculo, así que la captura y el render avanzan mientras la CPU infiere. El costo es la espera en cola: offline, con la cola siempre llena, la E2E p95 a 640 px sube a 126,9 ms. A ritmo de cámara las colas quedan casi vacías (espera p50 de 3,61 ms) y ese costo desaparece.
- **Menor resolución (640 a 320 px).** El costo de las convoluciones escala con el número de píxeles: un cuarto de píxeles bajó la inferencia de 18,0 a 5,5 ms por cuadro.
- **Formato ONNX.** ONNX Runtime ejecuta un grafo fijo y fusionado que reparte el cálculo entre los 10 núcleos (CPU media 491 % frente a 233 % del caso base): se gasta más CPU por segundo para producir cada cuadro mucho antes. En una prueba exploratoria a 640 px, ONNX fue más lento que PyTorch (46,6 frente a 52,8 FPS), por lo que solo se adopta a 320 px.
- **GPU (MPS) evaluada y descartada.** En una prueba exploratoria (calentamiento distinto, no comparable de forma estricta), MPS casi duplicó el rendimiento a 640 px (100,1 frente a 52,8 FPS, 1,9 veces), pero a 320 px igualó a la CPU (139,4 FPS): con un modelo tan pequeño, el costo fijo de enviar cada cuadro a la GPU y traer el resultado anula la ganancia de cálculo. La solución en CPU es igual de rápida y portable.
- **Lote de 1 y cola de 2.** Agrupar cuadros obligaría a esperar a que lleguen varios, lo que agrega latencia en streaming. Una cola de tamaño 2 limita la memoria y la espera; una cola sin límite crecería indefinidamente si la inferencia fuese más lenta que la fuente.
- **Sin reutilizar cajas viejas.** Se descartó dibujar las cajas de un cuadro anterior sobre el cuadro nuevo: eleva el número de FPS mostrado pero desalinea las anotaciones con objetos en movimiento. Aquí cada cuadro se muestra con su propia predicción.

6. Estabilidad y calidad de detección

Estabilidad: en el caso optimizado, el rendimiento por ventana de un segundo osciló entre 104 y 147 FPS (desviación 13,8); a ritmo de cámara, los tres casos mantuvieron 12,5 FPS con retraso de captura p95 igual a 0 ms. Entre corridas offline el caso de hilos a 640 px varió de 41,2 a 46,5 FPS, por lo que se reportan medianas. **Calidad:** síncrono e hilos a 640 px producen exactamente las mismas detecciones (0,40 por cuadro), porque procesan los mismos cuadros con el mismo modelo. A 320 px se obtienen 0,42 por cuadro. No se dispone de anotaciones de referencia, por lo que no se calcula mAP ni se afirma igualdad de calidad. La Figura 4 muestra dos límites reales: el modelo base COCO confunde autos vistos desde arriba con la clase cell phone en ambas resoluciones, y a 320 px se pierde el auto rojo en ese cuadro. Es decir, la optimización compra velocidad a cambio de sensibilidad a objetos pequeños, y para uso real convendría ajustar el modelo con imágenes cenitales etiquetadas.



Figura 4. Mismo cuadro (frame_id 220, el de más detecciones en el caso base) anotado por cada modelo; ejemplo cualitativo, no una métrica de calidad.

7. Limitaciones y conclusiones

Las cifras corresponden a un Apple M4 con un video corto; otro hardware dará otros valores. Las corridas en pantalla son una por caso y el render queda limitado por la sincronización de cv2.imshow (unos 12 ms por cuadro). En conclusión, la arquitectura de tres hilos con colas acotadas cumple el requisito de concurrencia sin carreras, y la combinación con menor resolución y ONNX supera los 60 FPS de forma estable con anotaciones alineadas a cada cuadro, a costa de una posible pérdida de detecciones que debe validarse con datos etiquetados.

Reproducción (código y resultados en https://github.com/AlejoMoncada/yolo-realtime-pipeline, carpeta benchmark-results/):

```
python export_model.py yolo11n.pt --imgsz 320
python benchmark.py car-detection.mp4 --device cpu --imgsz 640 --optimized-imgsz 320 --optimized-weights
yolo11n-320.onnx --repeats 3 [--realtime]
python pipeline.py car-detection.mp4 --mode threaded --weights yolo11n-320.onnx --imgsz 320 --device cpu --show
```